CKD Classification Assignment

1. Identify the problem

The objective is to predict Chronic Kidney Disease - CKD for individuals based on several parameters such as clinical and demographic features.

Problem Identification
Stage 1- Machine Learning
Stage 2 -What type of learning – Supervised Learning
Stage 3 –Classification – Binary classification with two outcome categories

2. Basic information about the dataset

The insurance_CKD.csv dataset contains 399 rows and 25 columns. Each row represents an individual. 24 columns are input features, and classification is the target variable. After dummy encoding, the notebook contains 28 columns, comprising 27 input features and classification_yes.

3. Pre-processing

I converted the categorical variables into indicator columns using pandas.get_dummies(dataset, drop_first=True). Examples in the notebook include rbc, pc, pcc, ba, htn, dm, cad, appet, pe and ane, as well as sg. These categories are nominal and do not have an inherent numerical order.

I applied standardization to place numerical features on a comparable scale. Standardization centers values around the mean and scales them by their standard deviation; it does not reduce the underlying differences between individuals.

4. Model development and comparison

I evaluated Support Vector Classification using SVC, Decision Tree Classification, Random Forest Classification, and Logistic Regression. Grid search compares parameter combinations using five cross-validation folds with the training data. The scores below are the reported best mean cross-validation scores.

Rank	Model	Best Mean CV score	Fold Standard Deviation
1	***Support Vector Classification***	**0.992520**	**0.009163**
2	Logistic Regression	0.988760	0.009179
3	Random Forest Classifier	0.984791	0.013772
4	Decision Tree Classifier	0.977463	0.027745

5. Detailed Experiment Results

Support Vector Classification

The SVM grid contains four kernels (linear, rbf, poly, and sigmoid), two gamma settings (auto, and scale) and five C values (10, 100, 1000, 2000, & 3000) giving 40 scenario combinations and 200 validation fits.

The best reported mean weighted F1 is 0.992520, with a fold standard deviation of 0.009163. Both C=10, kernel="sigmoid", gamma="auto" and the corresponding gamma="scale" combination have rank 1. GridSearchCV selected the combination with gamma='auto'; both combinations achieved the same best mean score.

mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_C	param_gamma	param_kernel	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_score
0.00413	0.001168	0.006595	0.001892	10	auto	linear	{'C': 10, 'gamma': 'auto', 'kernel': 'linear'}	0.981569	0.962636	0.962573	0.981031	0.981217	0.973805	0.009147	23
0.004692	0.001436	0.007756	0.001289	10	auto	rbf	{'C': 10, 'gamma': 'auto', 'kernel': 'rbf'}	0.963284	0.981014	1	1	1	0.988859	0.014751	4
0.0058	0.001342	0.006509	0.001595	10	auto	poly	{'C': 10, 'gamma': 'auto', 'kernel': 'poly'}	1	0.981014	1	1	0.981217	0.992446	0.009252	3
0.004218	0.001143	0.007634	0.001412	10	auto	sigmoid	{'C': 10, 'gamma': 'auto', 'kernel': 'sigmoid'}	0.981569	1	1	0.981031	1	0.99252	0.009163	1
0.004617	0.001022	0.006887	0.000685	10	scale	linear	{'C': 10, 'gamma': 'scale', 'kernel': 'linear'}	0.981569	0.962636	0.962573	0.981031	0.981217	0.973805	0.009147	23
0.004637	0.00087	0.006679	0.000835	10	scale	rbf	{'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}	0.963284	0.981014	1	1	1	0.988859	0.014751	4
0.00471	0.000735	0.006269	0.000628	10	scale	poly	{'C': 10, 'gamma': 'scale', 'kernel': 'poly'}	1	0.981014	1	0.981031	0.981217	0.988652	0.009266	6
0.004514	0.000744	0.005666	0.000664	10	scale	sigmoid	{'C': 10, 'gamma': 'scale', 'kernel': 'sigmoid'}	0.981569	1	1	0.981031	1	0.99252	0.009163	1
0.002449	0.000502	0.005767	0.001187	100	auto	linear	{'C': 100, 'gamma': 'auto', 'kernel': 'linear'}	0.981569	0.962636	0.962573	0.981031	0.981217	0.973805	0.009147	23
0.002099	0.000689	0.004046	0.001111	100	auto	rbf	{'C': 100, 'gamma': 'auto', 'kernel': 'rbf'}	0.963284	0.981014	1	0.981031	1	0.985066	0.013807	9
0.003539	0.000793	0.005526	0.000735	100	auto	poly	{'C': 100, 'gamma': 'auto', 'kernel': 'poly'}	0.963284	0.981014	1	1	0.981217	0.985103	0.013797	7
0.002714	0.000846	0.00451	0.000971	100	auto	sigmoid	{'C': 100, 'gamma': 'auto', 'kernel': 'sigmoid'}	0.981569	0.943699	0.944023	0.981031	1	0.970065	0.022459	33
0.002341	0.000767	0.004087	0.000894	100	scale	linear	{'C': 100, 'gamma': 'scale', 'kernel': 'linear'}	0.981569	0.962636	0.962573	0.981031	0.981217	0.973805	0.009147	23
0.002083	0.000605	0.004574	0.00068	100	scale	rbf	{'C': 100, 'gamma': 'scale', 'kernel': 'rbf'}	0.963284	0.981014	1	0.981031	1	0.985066	0.013807	9
0.003335	0.000914	0.003638	0.000692	100	scale	poly	{'C': 100, 'gamma': 'scale', 'kernel': 'poly'}	0.963284	0.981014	1	1	0.981217	0.985103	0.013797	7
0.00318	0.000856	0.004571	0.000783	100	scale	sigmoid	{'C': 100, 'gamma': 'scale', 'kernel': 'sigmoid'}	0.981569	0.943699	0.944023	0.981031	1	0.970065	0.022459	33
0.002553	0.000851	0.004863	0.000916	1000	auto	linear	{'C': 1000, 'gamma': 'auto', 'kernel': 'linear'}	0.981569	0.962636	0.962573	0.981031	0.981217	0.973805	0.009147	23
0.002178	0.000546	0.003152	0.000938	1000	auto	rbf	{'C': 1000, 'gamma': 'auto', 'kernel': 'rbf'}	0.963284	0.981014	1	0.981031	1	0.985066	0.013807	9
0.00219	0.000914	0.002532	0.002141	1000	auto	poly	{'C': 1000, 'gamma': 'auto', 'kernel': 'poly'}	0.963284	0.981014	0.981217	1	0.981217	0.981346	0.011613	17
0	0	0.003096	0.006191	1000	auto	sigmoid	{'C': 1000, 'gamma': 'auto', 'kernel': 'sigmoid'}	0.981569	0.943699	0.962573	0.981031	0.981217	0.970018	0.015021	35
0.003096	0.006191	0.006191	0.007583	1000	scale	linear	{'C': 1000, 'gamma': 'scale', 'kernel': 'linear'}	0.981569	0.962636	0.962573	0.981031	0.981217	0.973805	0.009147	23
0.003096	0.006191	0.003096	0.006191	1000	scale	rbf	{'C': 1000, 'gamma': 'scale', 'kernel': 'rbf'}	0.963284	0.981014	1	0.981031	1	0.985066	0.013807	9
0.003096	0.006191	0.006308	0.007725	1000	scale	poly	{'C': 1000, 'gamma': 'scale', 'kernel': 'poly'}	0.963284	0.981014	0.981217	1	0.981217	0.981346	0.011613	17
0.003154	0.006308	0.006308	0.007725	1000	scale	sigmoid	{'C': 1000, 'gamma': 'scale', 'kernel': 'sigmoid'}	0.981569	0.943699	0.962573	0.981031	0.981217	0.970018	0.015021	35
0	0	0.003174	0.006349	2000	auto	linear	{'C': 2000, 'gamma': 'auto', 'kernel': 'linear'}	0.981569	0.962636	0.962573	0.981031	0.981217	0.973805	0.009147	23
0.006413	0.007854	0.003174	0.006349	2000	auto	rbf	{'C': 2000, 'gamma': 'auto', 'kernel': 'rbf'}	0.963284	0.981014	1	0.981031	1	0.985066	0.013807	9
0	0	0.009081	0.00748	2000	auto	poly	{'C': 2000, 'gamma': 'auto', 'kernel': 'poly'}	0.963284	0.981014	0.981217	1	0.981217	0.981346	0.011613	17
0	0	0.006384	0.007819	2000	auto	sigmoid	{'C': 2000, 'gamma': 'auto', 'kernel': 'sigmoid'}	0.981569	0.943699	0.962573	0.981031	0.981217	0.970018	0.015021	35
0.006384	0.007819	0.006413	0.007854	2000	scale	linear	{'C': 2000, 'gamma': 'scale', 'kernel': 'linear'}	0.981569	0.962636	0.962573	0.981031	0.981217	0.973805	0.009147	23
0.003238	0.006476	0.003238	0.006476	2000	scale	rbf	{'C': 2000, 'gamma': 'scale', 'kernel': 'rbf'}	0.963284	0.981014	1	0.981031	1	0.985066	0.013807	9
0.005906	0.00729	0.003744	0.006239	2000	scale	poly	{'C': 2000, 'gamma': 'scale', 'kernel': 'poly'}	0.963284	0.981014	0.981217	1	0.981217	0.981346	0.011613	17
0.00321	0.006419	0.005906	0.00729	2000	scale	sigmoid	{'C': 2000, 'gamma': 'scale', 'kernel': 'sigmoid'}	0.981569	0.943699	0.962573	0.981031	0.981217	0.970018	0.015021	35
0	0	0.00321	0.006419	3000	auto	linear	{'C': 3000, 'gamma': 'auto', 'kernel': 'linear'}	0.981569	0.962636	0.962573	0.981031	0.981217	0.973805	0.009147	23
0	0	0.00321	0.006419	3000	auto	rbf	{'C': 3000, 'gamma': 'auto', 'kernel': 'rbf'}	0.963284	0.981014	1	0.981031	1	0.985066	0.013807	9
0.00321	0.006419	0.00321	0.006419	3000	auto	poly	{'C': 3000, 'gamma': 'auto', 'kernel': 'poly'}	0.963284	0.981014	0.981217	1	0.981217	0.981346	0.011613	17
0.008005	0.006536	0.006939	0.005228	3000	auto	sigmoid	{'C': 3000, 'gamma': 'auto', 'kernel': 'sigmoid'}	0.981569	0.943699	0.962573	0.981031	0.981217	0.970018	0.015021	35
0.000534	0.001068	0	0	3000	scale	linear	{'C': 3000, 'gamma': 'scale', 'kernel': 'linear'}	0.981569	0.962636	0.962573	0.981031	0.981217	0.973805	0.009147	23
0	0	0.003219	0.006437	3000	scale	rbf	{'C': 3000, 'gamma': 'scale', 'kernel': 'rbf'}	0.963284	0.981014	1	0.981031	1	0.985066	0.013807	9
0.006437	0.007884	0.009656	0.007884	3000	scale	poly	{'C': 3000, 'gamma': 'scale', 'kernel': 'poly'}	0.963284	0.981014	0.981217	1	0.981217	0.981346	0.011613	17
0.003219	0.006437	0	0	3000	scale	sigmoid	{'C': 3000, 'gamma': 'scale', 'kernel': 'sigmoid'}	0.981569	0.943699	0.962573	0.981031	0.981217	0.970018	0.015021	35

Decision Tree

The displayed grid contains two criteria (gini and entropy), three max_features settings (auto, sqrt, & log2) and two splitters (best, and random): 12 candidates, rather than 24.

The best valid combination is criterion="gini", max_features="log2", splitter="random", with mean validation score 0.977463 and standard deviation 0.027745.

Four rows using max_features="auto" have blank scores. Treat these as missing or failed evaluations, not scores of zero. Current DecisionTreeClassifier accepts "sqrt", "log2", None or numeric values; "auto" is unsupported.

mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_criterion	param_max_features	param_splitter	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_score
0.003181	0.006361	0	0	gini	auto	best	{'criterion': 'gini', 'max_features': 'auto', 'splitter': 'best'}								9
0.00205	0.004099	0	0	gini	auto	random	{'criterion': 'gini', 'max_features': 'auto', 'splitter': 'random'}								9
0.008971	0.006114	0.006127	0.005859	gini	sqrt	best	{'criterion': 'gini', 'max_features': 'sqrt', 'splitter': 'best'}	0.9451	0.942166	0.907035	0.942332	0.981031	0.943533	0.023426	7
0.005535	0.005867	0.015784	0.004555	gini	sqrt	random	{'criterion': 'gini', 'max_features': 'sqrt', 'splitter': 'random'}	0.963284	0.922185	0.962573	1	0.981217	0.965852	0.025793	4
0.004796	0.004335	0.010637	0.00219	gini	log2	best	{'criterion': 'gini', 'max_features': 'log2', 'splitter': 'best'}	0.944707	0.942166	0.869849	0.962264	0.981031	0.940004	0.037755	8
0.003316	0.00289	0.00935	0.002676	gini	log2	random	{'criterion': 'gini', 'max_features': 'log2', 'splitter': 'random'}	0.981569	0.924528	1	1	0.981217	0.977463	0.027745	1
0.001694	0.003387	0	0	entropy	auto	best	{'criterion': 'entropy', 'max_features': 'auto', 'splitter': 'best'}								9
0	0	0	0	entropy	auto	random	{'criterion': 'entropy', 'max_features': 'auto', 'splitter': 'random'}								9
0.003323	0.00407	0.005022	0.004101	entropy	sqrt	best	{'criterion': 'entropy', 'max_features': 'sqrt', 'splitter': 'best'}	0.981569	0.943699	0.907035	0.924528	0.962264	0.943819	0.026422	6
0.00459	0.004482	0.007092	0.004102	entropy	sqrt	random	{'criterion': 'entropy', 'max_features': 'sqrt', 'splitter': 'random'}	1	0.885265	0.962573	0.962264	0.981217	0.958264	0.03907	5
0.002465	0.00132	0.006955	0.003272	entropy	log2	best	{'criterion': 'entropy', 'max_features': 'log2', 'splitter': 'best'}	0.981569	0.942166	0.962264	0.981031	1	0.973406	0.019657	2
0.003965	0.003794	0.005762	0.004739	entropy	log2	random	{'criterion': 'entropy', 'max_features': 'log2', 'splitter': 'random'}	0.963284	0.943041	0.981031	0.981217	0.981217	0.969958	0.015134	3

Random_Forest

The displayed grid contains two criteria (gini and entropy), three max_features settings (auto, sqrt,and log2) and two tree counts (10 and 100), giving 12 candidates.

The best valid combination is criterion="gini", max_features="log2", n_estimators=100, with mean validation score 0.984791 and standard deviation 0.013772.

Four rows using max_features="auto" have blank scores. Current RandomForestClassifier does not accept this option.

mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_criterion	param_max_features	param_n_estimators	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_score
0.002372	0.004745	0	0	gini	auto	10	{'criterion': 'gini', 'max_features': 'auto', 'n_estimators': 10}								9
0.00042	0.000514	0	0	gini	auto	100	{'criterion': 'gini', 'max_features': 'auto', 'n_estimators': 100}								9
0.065258	0.007389	0.015563	0.000248	gini	sqrt	10	{'criterion': 'gini', 'max_features': 'sqrt', 'n_estimators': 10}	0.969697	0.957746	0.918033	0.969697	0.969697	0.956974	0.020013	8
0.559374	0.010516	0.045682	0.004079	gini	sqrt	100	{'criterion': 'gini', 'max_features': 'sqrt', 'n_estimators': 100}	0.969697	0.971429	0.984615	0.985075	0.985075	0.979178	0.007058	4
0.056593	0.007129	0.014908	0.002	gini	log2	10	{'criterion': 'gini', 'max_features': 'log2', 'n_estimators': 10}	0.969697	0.956522	0.96875	0.969697	1	0.972933	0.014425	6
0.535587	0.007221	0.0477	0.000374	gini	log2	100	{'criterion': 'gini', 'max_features': 'log2', 'n_estimators': 100}	1	0.985507	0.96875	0.969697	1	0.984791	0.013772	1
0.003232	0.006464	0	0	entropy	auto	10	{'criterion': 'entropy', 'max_features': 'auto', 'n_estimators': 10}								9
0.003223	0.006446	0	0	entropy	auto	100	{'criterion': 'entropy', 'max_features': 'auto', 'n_estimators': 100}								9
0.059768	0.00627	0.013917	0.00706	entropy	sqrt	10	{'criterion': 'entropy', 'max_features': 'sqrt', 'n_estimators': 10}	0.985075	0.971429	0.952381	0.985075	0.985075	0.975807	0.01285	5
0.466198	0.015461	0.046519	0.00232	entropy	sqrt	100	{'criterion': 'entropy', 'max_features': 'sqrt', 'n_estimators': 100}	1	0.957746	0.96875	0.985075	1	0.982314	0.016857	2
0.063846	0.000479	0.012648	0.006334	entropy	log2	10	{'criterion': 'entropy', 'max_features': 'log2', 'n_estimators': 10}	0.985075	0.957746	0.952381	0.969697	0.985075	0.969995	0.013529	7
0.397797	0.010414	0.025494	0.007766	entropy	log2	100	{'criterion': 'entropy', 'max_features': 'log2', 'n_estimators': 100}	1	0.957746	0.96875	0.969697	1	0.979239	0.017464	3

Logistic Regression Classification

I tested four solvers (newton-cg, lbfgs, liblinear, and sega) with L2 regularisation. Both newton-cg and lbfgs have rank 1, with the same mean validation score of 0.988760 and standard deviation 0.009179.

The highlighted newton-cg row is therefore a tied winner, not the only best solver.

mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_penalty	param_solver	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_score
0.096203	0.009844	0.014123	0.005022	l2	newton-cg	{'penalty': 'l2', 'solver': 'newton-cg'}	0.981569	0.981014	0.981217	1	1	0.98876	0.009179	1
0.066188	0.020806	0.006851	0.003426	l2	lbfgs	{'penalty': 'l2', 'solver': 'lbfgs'}	0.981569	0.981014	0.981217	1	1	0.98876	0.009179	1
0.012405	0.004347	0.013064	0.003568	l2	liblinear	{'penalty': 'l2', 'solver': 'liblinear'}	0.963284	0.981233	0.962573	0.981217	0.981217	0.973905	0.008965	4
0.034518	0.012331	0.006444	0.007899	l2	saga	{'penalty': 'l2', 'solver': 'saga'}	0.981569	0.981233	0.981217	0.981217	0.981217	0.981291	0.000139	3

6. Final Model and Justification

SVM has the highest numerical mean validation score in the supplied tables at 0.992520, followed by logistic regression at 0.988760, random forest at 0.984791 and decision tree at 0.977463. I therefore identify SVM as the provisional leading candidate.

Repository link: https://github.com/mmrousan/machine-learning-Classification-assignment